

Sprint 1 – System Design Document

TempTracker – GLOW Project, CSCC01

CRC Cards

Header	
Responsibility	Collaborators
• Primary tool for page navigation • Show/hide auth-specific menu items (e.g. displaying “Log Out” button only when logged in)	• BrowserClient • Next • useRouter

Map	
Responsibility	Collaborators
• Display interactive map using Leaflet • Show temperature data points on map • Handle map interactions (zoom, pan) • Show popup information for each temperature point	• Leaflet API • MapLegend

MapLegend	
Responsibility	Collaborators
• Display a legend for temperature ranges on the map	Self-contained

UploadTemperatureForm	
Parent: Form	
Responsibility	Collaborators
• Handle temperature data upload • Validate temperature inputs • Manage form state • Display form errors	• Sonner

LoginForm	
Parent: Form	
Responsibility	Collaborators
• Handle user authentication with login credentials • Handle authentication flow	• Sonner • BrowserClient • useRouter • Zod Schema Validation

RegisterForm	
Parent: Form	
Responsibility	Collaborators
• Handle user registration • Validate registration inputs • Create new user accounts	• Sonner • BrowserClient • useRouter • Zod Schema Validation

ThemeProvider	
Responsibility	Collaborators
• Manage application theme state • Handle system theme detection	Self-contained

Frequently used *Shadcn* components

Button	
Responsibility	Collaborators
• Render accessible button elements • Support multiple variants (default, ghost, link, etc.) • Support multiple sizes (sm, lg) • Manage loading states	• ThemeProvider

Input	
Responsibility	Collaborators
• Render form input elements	• ThemeProvider

• Handle different input types (text, password, email, number, etc.) • Handle focus/blur events	
--	--

Sonner (Toast)	
<i>Responsibility</i>	<i>Collaborators</i>
• Display toast notifications • Handle multiple notification types (e.g. success, error)	• ThemeProvider

Form	
Subclasses: UploadTemperatureForm, LoginForm, RegisterForm	
<i>Responsibility</i>	<i>Collaborators</i>
• Handle form state management • Provide form validation • Manage form submissions • Handle submission errors	• FormField • FormItem • FormLabel • FormDescription • FormMessage • ThemeProvider

Supabase

BrowserClient	
<i>Responsibility</i>	<i>Collaborators</i>
• Handle Supabase operations in client components • Provide interface for database interaction	Self-contained

ServerClient	
<i>Responsibility</i>	<i>Collaborators</i>
• Handle Supabase operations in server components • Provide interface for database interaction	Self-contained

Next.js

Link	
Responsibility	Collaborators
• Handle client-side navigation • Manage prefetching of pages	Self-contained

useRouter	
Responsibility	Collaborators
• Allow programmatic navigation • Handle route changes • Access route parameters	Self-contained

System Interactions and Assumptions

TempTracker's GLOW app is currently a mobile-first web app, used for checking water temperatures across the Great Lakes. TempTracker assumes that the user will be able to run this app directly through a hosted server in the future (Heroku, Vercel, GitHub Pages + hosted backend routes).

The app's full tech stack is described as follows:

- **Backend**
 - ExpressJS (currently minimal routes)
 - Supabase authentication and integration with Supabase Auth
- **Frontend**
 - NextJS using React
 - Tailwind V4 for styling
 - Shadcn for UI components
 - LeafletJS for map component (currently)
- **Database**
 - Using Supabase Storage and Realtime Database
 - Supabase hosts via PostgreSQL
- **Deployment**
 - To be determined

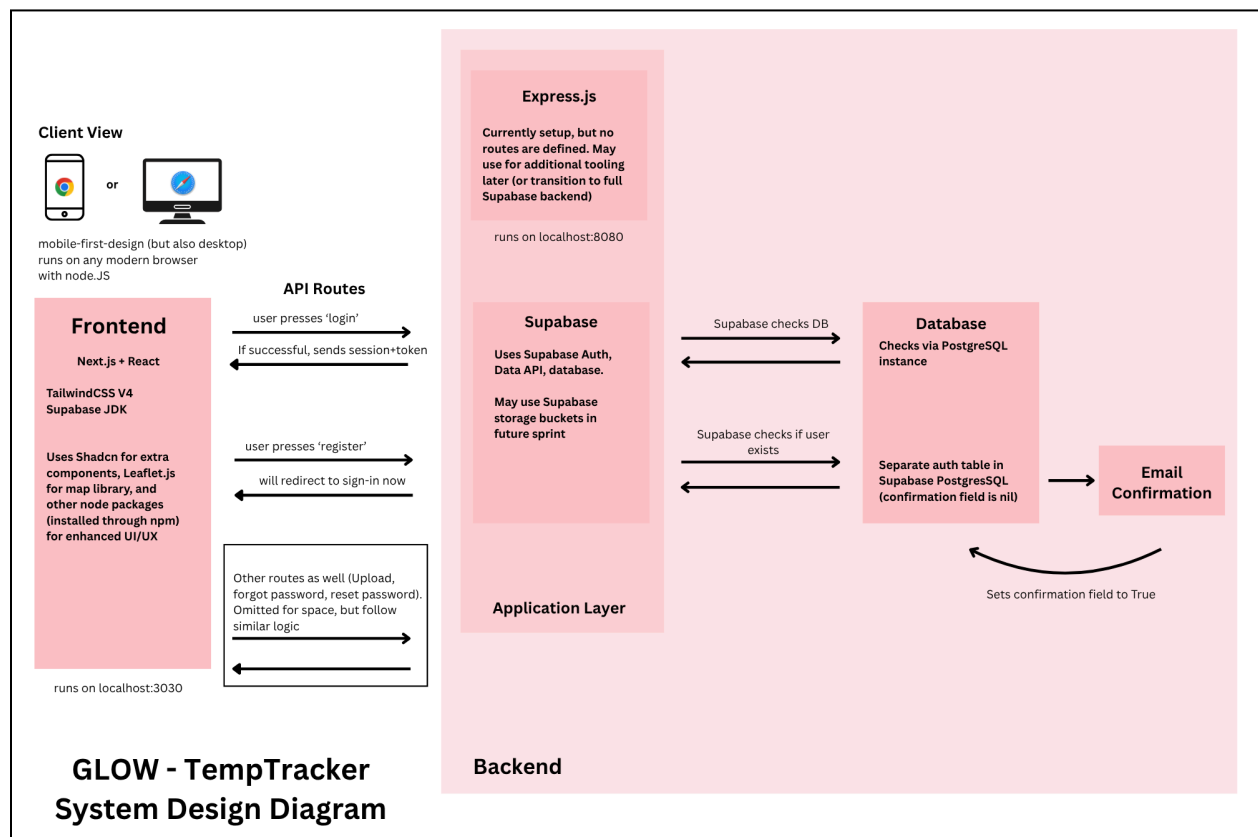
Currently, without a deployment, our GLOW app runs locally using npm and Node.js. Given the correct initialization of the app using *npm install* and *npm run dev* in both the client and server directories, the app can run on both the Windows and macOS operating systems. On localhost,

the aforementioned *npm install* command is assumed, as it installs all relevant libraries and node modules needed for the app to run.

Currently, the app doesn't heavily use the traditional ExpressJS routes and primarily uses Supabase (which acts as a backend-as-a-service). In future developments, we may create some ExpressJS routes to handle additional logic (upload data, etc).

The frontend currently runs on localhost port 3000 while the backend runs on localhost port 8080. Since the app will be hosted through a provider in the future, users don't need to worry about compiling it themselves. Any modern web browser with JavaScript enabled (Chrome, Firefox, Safari) can automatically compile using Node.js and its various tooling. OS restraints should also not be an issue, as modern browsers are equipped to handle all major operating systems.

System Architecture and Diagram



As previously mentioned, the tech stack uses Next.js and React for the frontend and Supabase, Express and PostgreSQL for the database and backend. The API integrations are shown in the image above, with some routes abstracted for clarity and to reduce repetition.

We chose Next.js with React for the frontend because React is the most prevalent and frequently updated frontend framework. It uses JSX syntax to allow JavaScript to run in component files, and Next.js allows each 'page' to be segregated, ensuring an easy-to-develop layout. To stay up to date and enforce the best UI/UX practices, we used TailwindCSS (V4) which is the most recent version of Tailwind, and supports new styling classes, themes, and utilities. We also imported additional libraries for extra components as well, including Shadcn for UI parts, and Leaflet.js for a map component.

For the backend, we wanted to focus on Supabase because of its speed and extensive features. As mentioned previously, Supabase is an alternative to Google Firebase and provides backend features (routes, database, authentication, storage), bundled together. Supabase also has a generous free tier, so hosting the backend will be less of an issue. Supabase also has great authentication features and lets us store hashed passwords, access users securely, send confirmation emails for registration, and have a separate user schema. Supabase integrates with PostgreSQL, which also works best, given that a relational database will work well later with tabular temperature data. Originally, we wanted to use Express.js for traditional app routes, but after concluding Sprint 1, we realised many of the tasks we want to implement may be possible entirely with Supabase's capabilities. Although not certain yet, this means we would not need to create traditional backend routes.